Data Structure

Assignment

Dept. of computer science

UG 1st Year (2nd Semester)



RAJENDRA UNIVERSITY

PRAJNA VIHAR,BALANGIR,767002

ODISHA

Submitted By.:

Rakesh Deep

Roll No.: BS24-086

Exam Roll No:- **R04524CSC017**

a) Write a program to perform various operation on a STACK (Insertion, deletion, display, view particular, modify).

- **Using Array**

```c
#include <stdio.h>
#define MAX_SIZE 100

int stack[MAX_SIZE];
int top = -1;

// Function to add element to stack
void push() {
    if (top >= MAX_SIZE - 1) {
        printf("\nStack is full! Cannot add more items.");
        return;
    }

    int new_item;
    printf("\nEnter the number to add: ");
    scanf("%d", &new_item);

    top++;
    stack[top] = new_item;
    printf("Item added successfully!");
}

// Function to remove element from stack
void pop() {
    if (top < 0) {
        printf("\nStack is empty! Nothing to remove.");
        return;
    }

    printf("\nRemoved item: %d", stack[top]);
    top--;
}

// Function to display all elements
void display() {
    if (top < 0) {
        printf("\nStack is empty!");
        return;
    }

    printf("\nStack items are:\n");
    for (int i = top; i >= 0; i--) {
        printf("%d\n", stack[i]);
    }
}
```

```c
// Function to view particular element
void view_item() {
    if (top < 0) {
        printf("\nStack is empty!");
        return;
    }

    int position;
    printf("\nEnter position (1 to %d): ", top + 1);
    scanf("%d", &position);

    if (position < 1 || position > top + 1) {
        printf("Invalid position!");
        return;
    }

    printf("Item at position %d is: %d", position, stack[top - position + 1]);
}

// Function to modify element
void modify() {
    if (top < 0) {
        printf("\nStack is empty!");
        return;
    }

    int position, new_value;
    printf("\nEnter position to modify (1 to %d): ", top + 1);
    scanf("%d", &position);

    if (position < 1 || position > top + 1) {
        printf("Invalid position!");
        return;
    }

    printf("Enter new value: ");
    scanf("%d", &new_value);

    stack[top - position + 1] = new_value;
    printf("Value modified successfully!");
}

int main() {
    int choice;

    while (1) {
        printf("\n\nSTACK OPERATIONS");
        printf("\n1. Add item (Push)");
        printf("\n2. Remove item (Pop)");
        printf("\n3. Display all items");
```

```
        printf("\n4. View specific item");
        printf("\n5. Modify item");
        printf("\n6. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: push(); break;
            case 2: pop(); break;
            case 3: display(); break;
            case 4: view_item(); break;
            case 5: modify(); break;
            case 6: return 0;
            default: printf("\nInvalid choice!");
        }
    }
    return 0;
}
```

- **Using Linked List**

```
#include <stdio.h>
#include <stdlib.h>

// Node structure for stack
struct Node {
    int data;
    struct Node* next;
};

// Function prototypes
struct Node* push(struct Node* top, int val);
struct Node* pop(struct Node* top);
void display(struct Node* top);
void viewParticular(struct Node* top, int pos);
void modify(struct Node* top, int pos, int val);

int main() {
    struct Node* top = NULL;
    int choice, val, pos;

    while (1) {
        printf("\nStack Operations Menu:\n");
        printf("1. Push\n2. Pop\n3. Display\n4. View Particular\n5. Modify\n6.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
```

```
            case 1: printf("Enter value to push: "); scanf("%d", &val); top = push(top,
val); break;
            case 2: top = pop(top); break;
            case 3: display(top); break;
            case 4: printf("Enter position to view: "); scanf("%d", &pos);
viewParticular(top, pos); break;
            case 5: printf("Enter position to modify: "); scanf("%d", &pos);
printf("Enter new value: "); scanf("%d", &val); modify(top, pos, val); break;
            case 6: exit(0);
            default: printf("Invalid choice!\n");
        } }
    return 0; }

// Push operation
struct Node* push(struct Node* top, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Stack overflow!\n");
        return top;
    }
    newNode->data = val;
    newNode->next = top;
    printf("Pushed %d successfully\n", val);
    return newNode;
}

// Pop operation
struct Node* pop(struct Node* top) {
    if (top == NULL) {
        printf("Stack underflow!\n");
        return NULL;
    }
    struct Node* temp = top;
    printf("Popped %d successfully\n", top->data);
    top = top->next;
    free(temp);
    return top;
}

// Display operation
void display(struct Node* top) {
    if (top == NULL) { printf("Stack is empty!\n"); return; }
    printf("Stack elements: ");
    struct Node* temp = top;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

```c
// View particular element
void viewParticular(struct Node* top, int pos) {
    if (top == NULL) {
        printf("Stack is empty!\n");
        return;
    }
    struct Node* temp = top;
    int count = 1;
    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }
    if (temp == NULL) {
        printf("Position not found!\n");
    } else {
        printf("Element at position %d is: %d\n", pos, temp->data);
    }
}

// Modify element
void modify(struct Node* top, int pos, int val) {
    if (top == NULL) {
        printf("Stack is empty!\n");
        return;
    }
    struct Node* temp = top;
    int count = 1;
    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }
    if (temp == NULL) {
        printf("Position not found!\n");
    } else {
        temp->data = val;
        printf("Modified successfully!\n");
    }
}
```

b) Write a program to perform various operation on a QUEUE (Insertion, deletion, display, view particular, modify).

- **Using Array**

```c
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1;
```

```c
int rear = -1;

void insert() {
    int count, val;
    printf("How many elements do you want to insert? ");
    scanf("%d", &count);

    if (rear + count >= MAX) {
        printf("Cannot insert %d elements. Queue capacity exceeded!\n", count);
        return;
    }

    printf("Enter %d values separated by spaces: ", count);
    for (int i = 0; i < count; i++) {
        scanf("%d", &val);
        if (front == -1)
            front = 0;
        rear++;
        queue[rear] = val;
    }
    printf("Insertion successful!\n");
}

void delete() {
    if (front == -1 || front > rear) {
        printf("Queue is empty!\n");
        return;
    }
    printf("Deleted element is: %d\n", queue[front]);
    front++;
    if (front > rear)
        front = rear = -1;
}

void display() {
    if (front == -1 || front > rear) {
        printf("Queue is empty!\n");
        return;
    }
    printf("Queue elements are:\n");
    for (int i = front; i <= rear; i++)
        printf("%d ", queue[i]);
    printf("\n");
}
void viewParticular() {
    int pos;
    printf("Enter position (0 to %d): ", rear - front);
    scanf("%d", &pos);
    if (front == -1 || front + pos > rear) {
        printf("Invalid position!\n");
```

```
        return;
    }
    printf("Element at position %d is: %d\n", pos, queue[front + pos]);
}

void modify() {
    int pos, val;
    printf("Enter position (0 to %d): ", rear - front);
    scanf("%d", &pos);
    if (front == -1 || front + pos > rear) {
        printf("Invalid position!\n");
        return;
    }
    printf("Enter new value: ");
    scanf("%d", &val);
    queue[front + pos] = val;
    printf("Modification successful!\n");
}

int main() {
    int choice;
    while (1) {
        printf("\n1. Insert\n2. Delete\n3. Display\n4. View Particular\n5. Modify\n6.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: insert(); break;
            case 2: delete(); break;
            case 3: display(); break;
            case 4: viewParticular(); break;
            case 5: modify(); break;
            case 6: return 0;
            default: printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

- **Using Linked List**

```
#include <stdio.h>
#include <stdlib.h>

// Structure for queue node
struct Node {
    int data;
```

```
    struct Node* next;
};

struct Node* front = NULL;
struct Node* rear = NULL;

// Function to insert element in queue (enqueue)
void enqueue() {
    int n, i, value;
    printf("Enter number of elements to insert: ");
    scanf("%d", &n);

    printf("Enter %d elements separated by space: ", n);
    for(i = 0; i < n; i++) {
        scanf("%d", &value);

        struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
        if (newNode == NULL) {
            printf("Memory allocation failed!\n");
            return;
        }

        newNode->data = value;
        newNode->next = NULL;

        if (front == NULL) {
            front = rear = newNode;
        } else {
            rear->next = newNode;
            rear = newNode;
        }
    }
    printf("%d elements inserted successfully!\n", n);
}

// Function to delete element from queue (dequeue)
void dequeue(){
    if (front == NULL) {
        printf("Queue is empty!\n");
        return;
    }

    struct Node* temp = front;
    front = front->next;
    printf("Deleted element: %d\n", temp->data);
    free(temp);

    if (front == NULL) {
        rear = NULL;
    }
```

```
}

// Function to display queue elements
void display() {
    if (front == NULL) {
        printf("Queue is empty!\n");
        return;
    }

    struct Node* temp = front;
    printf("Queue elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

// Function to view particular element
void viewParticular() {
    if (front == NULL) {
        printf("Queue is empty!\n");
        return;
    }

    int pos, count = 1;
    printf("Enter position to view: ");
    scanf("%d", &pos);

    struct Node* temp = front;
    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) {
        printf("Position not found!\n");
    } else {
        printf("Element at position %d is: %d\n", pos, temp->data);
    }
}

// Function to modify element
void modify() {
    if (front == NULL) {
        printf("Queue is empty!\n");
        return;
    }

    int pos, newValue, count = 1;
```

```
    printf("Enter position to modify: ");
    scanf("%d", &pos);

    struct Node* temp = front;
    while (temp != NULL && count < pos) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL)
        printf("Position not found!\n");
    else {
        printf("Enter new value: ");
        scanf("%d", &newValue);
        temp->data = newValue;
        printf("Element modified successfully!\n");
    }
}

int main() {
    int choice;

    while (1) {
        printf("\n1. Insert\n2. Delete\n3. Display\n4. View Particular\n5. Modify\n6.
Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: enqueue(); break;
            case 2: dequeue(); break;
            case 3: display(); break;
            case 4: viewParticular(); break;
            case 5: modify(); break;
            case 6: printf("Exiting program.\n"); exit(0);
            default: printf("Invalid choice!\n");
        }
    }
    return 0;
}
```